



UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales

Electrónica Digital II

Informe de Laboratorio Nº3

Alumnos:
Gimenez, Galilea Candelaria
Ramos, Antonella Carolina
Salomon Oliden, Naiara Luz
Valencia Pezo, Luz Estrella

Córdoba,

1. Introducción

El objetivo de este trabajo práctico es diseñar e implementar un contador decimal de 00 a 99 utilizando el microcontrolador PIC16F887. El sistema detecta cada pulsación de un botón en el pin RE2, incrementa la cuenta y vuelve a 00 al superar el valor 99. El estado del contador se muestra simultáneamente en dos displays de 7 segmentos multiplexados y en un banco de 8 LEDs conectado al Puerto D para verificar el valor en binario.

A nivel de software y hardware, se configuraron los pines como digitales, deshabilitando las funciones analógicas. La visualización se realiza mediante multiplexado por división de tiempo, alternando el encendido de las unidades y decenas cada 3 ms para aprovechar la persistencia visual y evitar parpadeos. Para la lógica de control, el valor binario se convierte a decimal restando 10 de forma sucesiva y se decodifica utilizando una tabla de 7 segmentos (`TABLA_7SEG`). Para el control del pulsador, se implementó un mecanismo de antirrebote por software que combina la variable `ESTADO_BOTON`, encargada de memorizar si una pulsación ya fue registrada, con la variable `DEB_CONT`, que permite confirmar la estabilidad de la señal mediante cuatro muestras consecutivas tanto al presionar como al liberar el botón. De esta manera, se busca evitar incrementos múltiples ocasionados por los rebotes mecánicos sin introducir retardos bloqueantes prolongados que interrumpan el refresco de los displays.

2. Hardware

2.1. Asignación y justificación de pines

Módulo	Señal	Terminal del PIC
Comunicación serie	TX	RC6
Comunicación serie	RX	RC7
Pulsador	Entrada	RE2
Banco de LED	LED 0 a LED 7	RD0 a RD7
Displays	Segmentos a a g y dp	RC0 a RC5, RE0, RE1
Displays	Selección de dígitos	RA0, RA1, RA2, RA3
Teclado	Filas	RB4, RB5, RB6, RB7
Teclado	Columnas	RB0, RB1, RB2, RB3
Conversor ADC	Entrada analógica	RA5 (AN4)

Tabla 1: *Asignación de terminales.*

Función	Pin	Configuración
Pulsador	10 (RE2)	Entrada digital
LED0	19 (RD0)	Salida digital
LED1	20 (RD1)	Salida digital
LED2	21 (RD2)	Salida digital
LED3	22 (RD3)	Salida digital
LED4	27 (RD4)	Salida digital
LED5	28 (RD5)	Salida digital
LED6	29 (RD6)	Salida digital
LED7	30 (RD7)	Salida digital
SEG-a_4	15 (RCO)	Salida digital
SEG-b_5	16 (RC1)	Salida digital
SEG-c_8	17 (RC2)	Salida digital
SEG-d_7	18 (RC3)	Salida digital
SEG-e_6	23 (RC4)	Salida digital
SEG-f_2	24 (RC5)	Salida digital
SEG-g_1	8 (RE0)	Salida digital
dp_9	9 (RE1)	Salida digital
BASE NPN D0	2 (RA0)	Salida digital
BASE NPN D1	3 (RA1)	Salida digital

Tabla 2: *Asignación de pines utilizada.*

Justificación de la elección de pines:

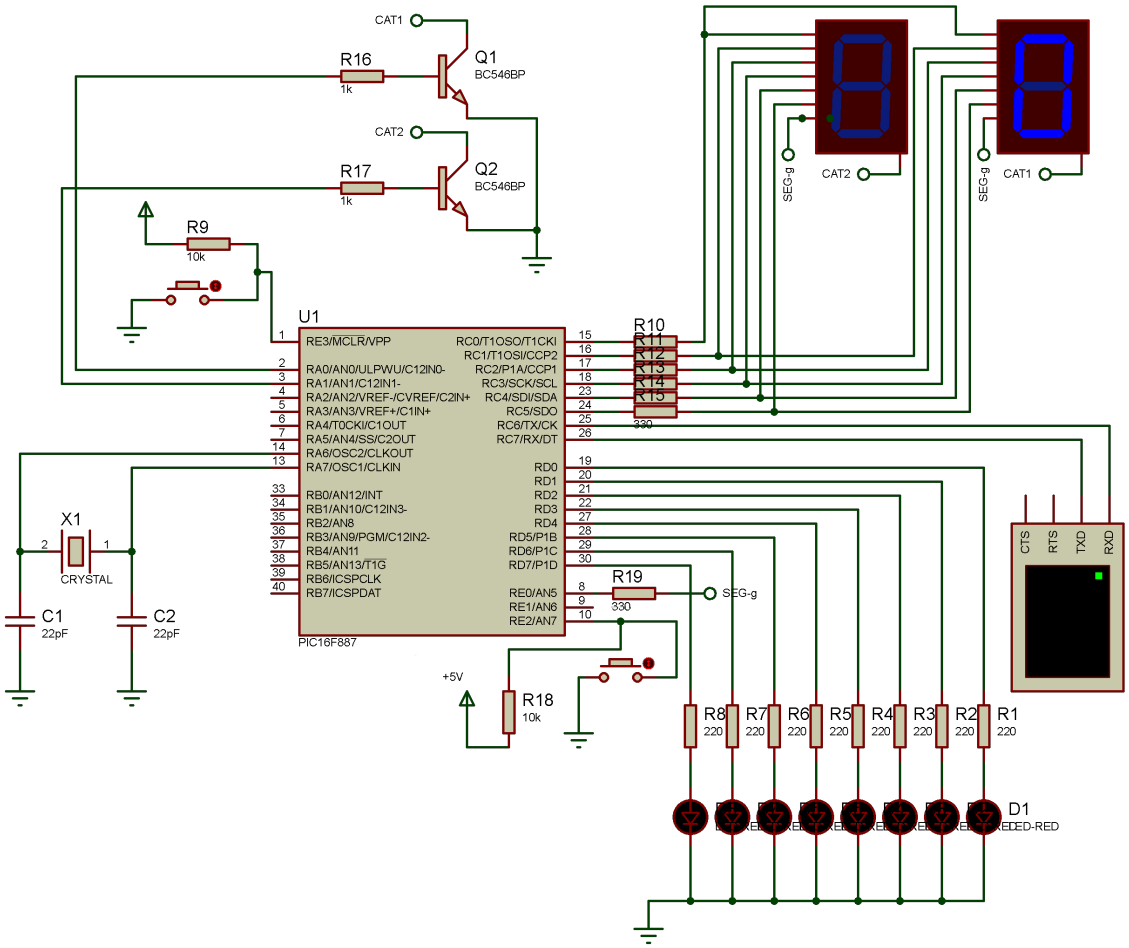
Puerto D (RD0:RD7): Se eligió como puerto de salida digital completo para controlar el banco de 8 LEDs. Al ser un puerto de 8 bits sin funciones analógicas compartidas conflictivas.

Pin RE2: Se eligió como entrada digital para el pulsador, recomendación para mejor organización.

Puerto C (RC0:RC5) y Puerto E (RE0:RE1): El control de los siete segmentos más el punto decimal (dp), de los displays, requiere ocho líneas de datos, para mejor organización se decidió utilizar el puerto C, sin embargo como él mismo tiene los pines RC6 y RC7 bloqueados por la comunicación serie, se utilizan los seis pines restantes y se completa el bus tomando los pines RE0 y RE1 del puerto E.

Puerto A (RA0:RA3): Se utiliza el puerto A (desactivando previamente su conversor analógico-digital) porque permite manejar señales de control individuales sin desperdiciar puertos completos de 8 bits. Estos pines envían la señal lógica (5V) a las bases de los transistores NPN para ejecutar el multiplexado.

2.2. Diagrama de conexión



2.3. Cálculos eléctricos

2.3.1 Resistencias para led

Para limitar la corriente que circula a través de los LEDs, se calcula la resistencia necesaria mediante la siguiente expresión:

$$R = \frac{V_{CC} - V_F}{I_F}$$

Se consideran los siguientes parámetros:

- Tensión de alimentación: $V_{CC} = 5\,\text{V}$

- Caída de tensión del LED rojo: $V_F = 2\,\text{V}$
- Corriente deseada: $I_F = 0{,}014\,\text{A}$ (14 mA), seleccionada para asegurar un encendido visible sin sobrecargar las salidas del microcontrolador.

Sustituyendo:

$$R = \frac{5\,\text{V} - 2\,\text{V}}{0{,}014\,\text{A}} \approx 214{,}29\,\Omega \implies R \approx 220\,\Omega$$

Por lo tanto, se adopta el valor comercial estándar más próximo de $220\,\Omega$.

Con dicha resistencia, la corriente teórica efectiva es:

$$I = \frac{5\,\text{V} - 2\,\text{V}}{220\,\Omega} \approx 13{,}64\,\text{mA}$$

Este valor cumple con las especificaciones del PIC16F887, cuyo límite máximo absoluto por pin de E/S es de 25 mA y de corriente combinada para los puertos C y D (máximo de 200 mA). C

2.3.2 Cálculo de las resistencias limitadoras de los segmentos (R_{seg})

Para limitar la corriente que circula a través de cada uno de los segmentos del display y garantizar un brillo adecuado sin comprometer la integridad del microcontrolador, se calcularon y verificaron las resistencias en serie utilizando un valor comercial de $R_{\text{seg}} = 330\,\Omega$.

Parámetros considerados

- Tensión de alimentación: $V_{CC} = 5\,\text{V}$.
- Tensión directa del segmento (LED rojo estándar): $V_D \approx 2{,}0\,\text{V}$.
- Tensión colector-emisor de saturación del transistor de habilitación: $V_{CE(\text{sat})} \approx 0{,}2\,\text{V}$.
- Resistencia limitadora seleccionada: $R_{\text{seg}} = 330\,\Omega$.

Cálculo de corriente por segmento

Aplicando la ley de Ohm en la malla correspondiente a un segmento encendido:

$$I_{\text{seg}} = \frac{V_{CC} - V_D - V_{CE(\text{sat})}}{R_{\text{seg}}} \tag{1}$$

Sustituyendo los valores numéricos:

$$I_{\text{seg}} = \frac{5\,\text{V} - 2{,}0\,\text{V} - 0{,}2\,\text{V}}{330\,\Omega} = \frac{2{,}8\,\text{V}}{330\,\Omega} \approx 8{,}48\,\text{mA} \tag{2}$$

Verificación eléctrica

- **Límite por terminal del microcontrolador:** La corriente demandada (8,48 mA) es inferior a la corriente máxima por terminal permitida por el PIC16F887 ($I_{\text{máx}} = 25\,\text{mA}$).
- **Consumo y disipación:** La potencia disipada por cada resistencia es:

$$P_R = I_{\text{seg}}^2 \cdot R_{\text{seg}} = (8{,}48\,\text{mA})^2 \cdot 330\,\Omega \approx 23{,}7\,\text{mW} \tag{3}$$

  Valor ampliamente cubierto por resistencias estándar de 1/4 W (250 mW).

2.3.3 Dimensionamiento de excitación (Transistores de selección)

Dado que los displays empleados son de **cátodo común**, la habilitación de cada dígito se efectúa cerrando el circuito hacia masa a través de un transistor de conmutación NPN debido a que conduce corriente desde su colector hacia su emisor (modelo BC548BP, identificado en el esquema como Q_1 y Q_2) controlado desde los terminales de control del microcontrolador mediante resistencias de base $R_{16} = R_{17} = 1\,\text{k}\Omega$.

Cálculo y dimensionamiento de la resistencia de base (R_B)

Para asegurar que los transistores de selección (Q_1 y Q_2, modelo BC548BP) operen en la región de **saturación profunda** cuando se habilita el dígito correspondiente, se dimensiona la resistencia de base R_B (R_{16} y R_{17}) de modo que la caída colector-emisor sea despreciable ($V_{CE(\text{sat})} \leq 0{,}2\,\text{V}$).

1. **Corriente máxima de colector:** Considerando los 7 segmentos encendidos simultáneamente:

$$I_{C,\text{máx}} = 7 \cdot I_{\text{seg}} = 7 \cdot 8{,}48\,\text{mA} \approx 59{,}36\,\text{mA} \tag{4}$$

2. **Corriente de base requerida para saturación:** Para garantizar una conmutación robusta como interruptor saturado, se adopta un factor de ganancia forzada típico de diseño $\beta_{\text{forzado}} = h_{FE(\text{sat})} \approx 15$ (dentro del rango estándar recomendado de 10 a 20):

$$I_{B,\text{mín}} = \frac{I_{C,\text{máx}}}{h_{FE(\text{sat})}} = \frac{59{,}36\,\text{mA}}{15} \approx 3{,}96\,\text{mA} \tag{5}$$

3. **Cálculo de la resistencia de base teórica:** Considerando la tensión en nivel alto entregada por el puerto del microcontrolador ($V_{OH} \approx 4{,}5\,\text{V}$) y la tensión directa base-emisor en saturación ($V_{BE(\text{sat})} \approx 0{,}7\,\text{V}$):

$$R_{B,\text{teórica}} \leq \frac{V_{OH} - V_{BE(\text{sat})}}{I_{B,\text{mín}}} = \frac{4{,}5\,\text{V} - 0{,}7\,\text{V}}{3{,}96\,\text{mA}} = \frac{3{,}8\,\text{V}}{3{,}96\,\text{mA}} \approx 959{,}6\,\Omega \tag{6}$$

4. **Elección del valor normalizado y verificación:** Se adopta el valor comercial estándar más próximo:

$$R_B = 1\,\text{k}\Omega \tag{7}$$

Con este resistor, la corriente real inyectada por el terminal del PIC16F887 es:

$$I_B = \frac{4{,}5\,\text{V} - 0{,}7\,\text{V}}{1\,\text{k}\Omega} = \frac{3{,}8\,\text{V}}{1000\,\Omega} = 3{,}8\,\text{mA} \tag{8}$$

La ganancia forzada resultante es:

$$h_{FE(\text{forzado})} = \frac{I_{C,\text{máx}}}{I_B} = \frac{59{,}36\,\text{mA}}{3{,}8\,\text{mA}} \approx 15{,}6 \tag{9}$$

Como $h_{FE(\text{forzado})} < 20$, se garantiza la saturación profunda del transistor. Además, los 3,8 mA demandados al terminal del microcontrolador se encuentran muy por debajo del límite de corriente admisible (25 mA).

Corriente máxima por dígito ($I_{C,\text{máx}}$)

La máxima corriente que debe absorber el terminal común ocurre en la condición más desfavorable, es decir, cuando se iluminan los siete segmentos del dígito "8":

$$I_{C,\text{máx}} = 7 \cdot I_{\text{seg}} = 7 \cdot 8{,}48\,\text{mA} \approx 59{,}36\,\text{mA} \tag{10}$$

2.3.4 Verificacion de limites de corriente del PIC16F887

De acuerdo con la hoja de datos del PIC16F887, la corriente máxima absoluta admitida por un terminal de entrada/salida es de 25 mA. Puesto que $I_{C,\text{máx}} = 59{,}36\,\text{mA} > 25\,\text{mA}$, conectar el cátodo común directamente a un pin del microcontrolador provocaría una sobrecorriente con riesgo de daño permanente o mal funcionamiento del puerto. Por ende, se justifica plenamente el uso de la etapa con transistores NPN en emisor común.

3. Software

3.1. Descripción del algoritmo

El programa desarrollado tiene como objetivo controlar un contador binario y decimal de 00 a 99. Su estructura se organiza alrededor de cuatro etapas principales: la preparación inicial del chip, la lectura del pulsador, el procesamiento del número y el encendido de los LEDs y displays.

1. Configuración inicial: Al encender el circuito, el programa desactiva las funciones analógicas para asegurar que todos los pines trabajen exclusivamente en modo digital. A continuación, define cuales pines y/o puertos actuarán como salidas (para los segmentos de los displays, los transistores y el bus de LEDs) y cual como entrada (el botón en RE2). Para terminar la preparación, limpia la memoria y fija el contador en 00 para evitar destellos o valores erróneos en el arranque.

2. Lectura del botón y filtro antirrebote: El microcontrolador revisa constantemente el estado del pulsador conectado a RE2. Debido a que los contactos mecánicos pueden producir transiciones rápidas entre los niveles lógicos alto y bajo, no se considera válida una pulsación a partir de una única lectura. Para mejorar la detección, se agrego y utiliza la variable `DEB_CONT`, que cuenta cuatro muestras consecutivas del nivel bajo para confirmar la presión y cuatro muestras consecutivas del nivel alto para confirmar la liberación. Por su parte, `ESTADO_BOTON` memoriza si la pulsación ya fue registrada, evitando incrementar repetidamente el contador mientras el botón permanece presionado. Si la señal cambia antes de completar las muestras requeridas, la confirmación se reinicia. Este procedimiento se integra al ciclo principal del programa para mantener el refresco de los displays multiplexados.

3. Cálculo del número y actualización de LEDs: Cada vez que se confirma una pulsación válida, el programa le suma 1 al contador y verifica si alcanzó el valor 100; de ser así, lo reinicia automáticamente en 00. Este valor se envía directamente al banco de LEDs para mostrar el total en formato binario. En paralelo, una rutina matemática convierte ese mismo número a formato decimal, separándolo en decenas y unidades mediante restas consecutivas de 10.

4. Visualización en los displays (multiplexado): Para controlar ambos displays de 7 segmentos compartiendo los mismos pines de datos, se aplica la técnica de multiplexado por división de tiempo. Primero se apagan ambos displays y

se envían los segmentos correspondientes a las unidades, habilitando D0 durante aproximadamente 3 ms. Luego se apagan nuevamente, se envían los segmentos de las decenas y se habilita D1 durante otros 3 ms. Este ciclo se repite continuamente. Después de cada ciclo de visualización, el programa consulta el pulsador y actualiza el contador únicamente cuando se confirma una nueva pulsación válida. De esta manera, el programa combina la visualización multiplexada con la comprobación del antirrebote.

3.2. Diagrama de flujo

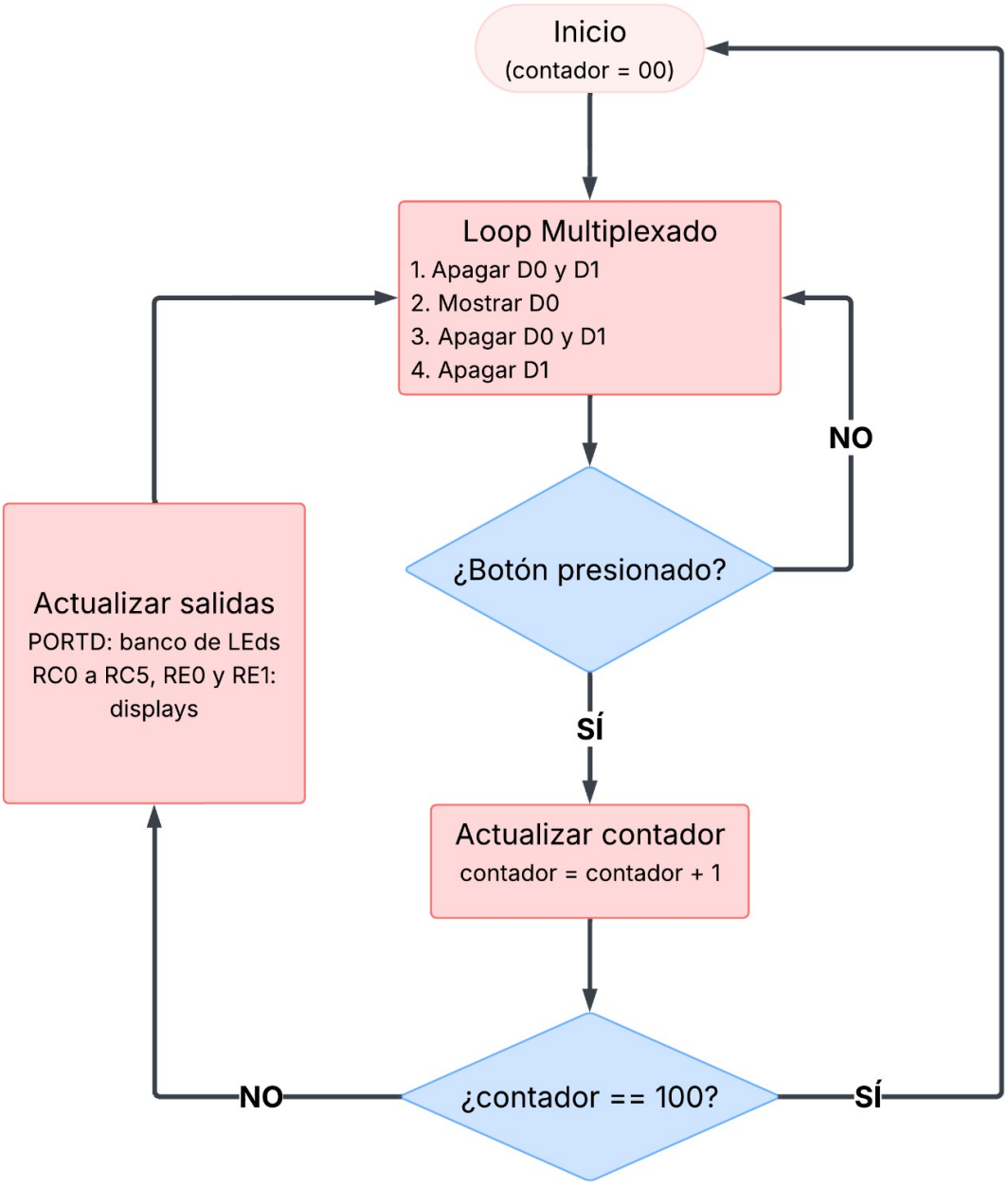


Figura 1: *Diagrama de flujo*

3.3. Análisis temporal de rutinas

Para el PIC16F887, considerando el cristal de 4 MHz utilizado en el trabajo, determinar primero el período del ciclo de instrucción mediante

$$T_{CY} = \frac{4}{F_{OSC}}.$$

$$T_{CY} = \frac{4}{4 \times 10^6 \text{ Hz}} = 1 \ \mu\text{s}$$

Cada ciclo de instruction toma exactamente

$$1 \ \mu\text{s}$$

Las instrucciones comunes tardan 1 ciclo (1 μs), mientras que las instrucciones que producen un salto de flujo tardan dos ciclos (2 μs).

Segmento `RETARDO_3MS`:

Se analiza el tiempo exacto de ejecución de la subrutina de retardo de 3 milisegundos utilizada para el refresco de los displays multiplexados. El tiempo de esta rutina depende puramente del código y no de botones externos.

```
            CALL    RETARDO_3MS   ; [2 ciclos]
; ------------------------------------------
RETARDO_3MS:
            MOVLW   d'3'    ; [1 ciclo]
            MOVWF   REG_MS  ; [1 ciclo]
loop_ms:
            MOVLW   d'249'  ; [1 ciclo]
            MOVWF   REG_US  ; [1 ciclo]
loop_us:
            NOP             ; [1 ciclo]
            DECFSZ  REG_US, F   ; [1 ciclo si no salta, 2 ciclos si salta]
            GOTO    loop_us     ; [2 ciclos]
            DECFSZ  REG_MS, F   ; [1 ciclo si no salta, 2 ciclos si salta]
            GOTO    loop_ms     ; [2 ciclos]
            RETURN              ; [2 ciclos]
```

Cálculo detallado de ciclos

1. **Configuración inicial (fuera de los bucles):**

- `CALL RETARDO_3MS`: 2 ciclos.
- `MOVLW d'3'`: 1 ciclo.
- `MOVWF REG_MS`: 1 ciclo.

Subtotal inicial: $1 + 2 + 1 = 4$ ciclos.

2. **Bucle interno de microsegundos (`loop_ms`). Genera 1 ms:**
   - Carga de variables: `MOVLW d'249'` (1 ciclo) + `MOVWF REG_US` (1 ciclo) = 2 ciclos.
   - Iteraciones normales (248 iteraciones): `NOP` (1 ciclo) + `DECFSZ` sin salto (1 ciclo) + `GOTO` (2 ciclos). = 4 ciclos $\times$ 248 = 992 ciclos.
   - Última iteración (1 iteración): La variable llega a cero. Se ejecuta `NOP` (1 ciclo) + `DECFSZ` salta al completarse el decremento, insumiendo 2 ciclos. El `GOTO` ya no se ejecuta. = 3 ciclos $\times$ 1 = 3 ciclos.
   - Subtotal por cada milisegundo: 2 (carga) + 992 (normales) + 3 (última) = 997 ciclos.
3. **Bucle externo de milisegundos(`loop_ms`). Repite 3 veces:**
   - Primeras 2 iteraciones (`REG_MS` pasa de 3 a 2, y de 2 a 1): Se ejecuta el bucle interno completo (997 ciclos) + `DECFSZ` sin salto (1 ciclo) + `GOTO loop_ms` (2 ciclos).

$$2 \times (997 + 1 + 2) = 2 \times 1000 = 2000 \text{ ciclos}$$

   - Última iteración (`REG_MS` llega a 0): Se ejecuta el bucle interno (997 ciclos) + `DECFSZ` con salto (2 ciclos).

$$997 + 2 = 999 \text{ ciclos}$$

Subtotal de bucles:

$$2000 + 999 = 2999 \text{ ciclos}$$

4. **Retorno de subrutina:**
   - `RETURN`: 2 ciclos.

Tiempo total obtenido

Sumando todas las partes del código analizado:

$$\text{Ciclos totales} = 4 \text{ (inicio)} + 2999 \text{ (bucles)} + 2 \text{ (retorno)} = 3005 \text{ ciclos}$$

Multiplicando por el período del ciclo de instrucción ($T_{CY} = 1\ \mu$s):

$$T_{\text{total}} = 3005 \times 1\ \mu\text{s} = 3005\ \mu\text{s} = 3{,}005 \text{ ms}$$

3.4. Código fuente

```
list p=16f887
    #include <p16f887.inc>

    __CONFIG _CONFIG1, _FOSC_XT & _WDTE_OFF & _PWRTE_ON & _MCLRE_ON & _LVP_OFF
    __CONFIG _CONFIG2, _BOR_OFF

; variables
contador     EQU     0x20        ; guarda el valor en binario
unidades     EQU     0x21        ; para el 1er display
decenas      EQU     0x22        ; para el 2do display
ESTADO_BOTON EQU     0x23        ; memoriza el estado de la pulsacion
PATRON_TEMP  EQU     0x24        ; para el mapeo de los 7 segmentos
PORTC_TEMP   EQU     0x25        ; conservar otras salidas del puerto C
PORTE_TEMP   EQU     0x26        ; preparar las salidas del puerto E
REG_MS       EQU     0x27        ; retardo en milisegundos
REG_US       EQU     0x28        ; retardo interno
DEB_CONT     EQU     0x29        ; contador de muestras antirrebote

; vector de reset
    ORG     0x00
    GOTO    main

; vector de interrupcion
    ORG     0x0004
    RETFIE

    ORG     0x0020

main:
    BANKSEL ANSEL
    CLRF    ANSEL               ; AN0-AN7 a digital (desactiva AN7 en RE2)
    CLRF    ANSELH

    BANKSEL TRISA
    BCF     TRISA, 0            ; RA0 como salida: habilita D0 (unidades)
    BCF     TRISA, 1            ; RA1 como salida: habilita D1 (decenas)

    MOVLW   b'11000000'
    MOVWF   TRISC               ; RC0-RC5 como salidas (segmentos a-f)
```

```
    CLRF    TRISD               ; PORTD como salida (LEDs)

    MOVLW   b'11111100'         ; RE0 y RE1 salidas (g y dp), RE2 entrada (pulsador
    MOVWF   TRISE
    BSF     TRISE, 2            ; RE2 como entrada

    ; inicializar puertos y variables en 0
    BANKSEL PORTA
    CLRF    PORTA
    CLRF    PORTC
    CLRF    PORTD
    CLRF    PORTE

    BANKSEL contador
    CLRF    contador
    CLRF    unidades
    CLRF    decenas
    CLRF    ESTADO_BOTON
    CLRF    DEB_CONT

    ; iniciar contador en 00
    CALL    Convertir_Bin_Dec
    MOVF    contador, W
    MOVWF   PORTD

Loop_Multiplexado:
    ; 1. apagar displays
    BANKSEL PORTA
    BCF     PORTA, 0
    BCF     PORTA, 1

    ; 2. mostrar unidades
    BANKSEL unidades
    MOVLW   HIGH TABLA_7SEG
    MOVWF   PCLATH
    MOVF    unidades, W
    CALL    TABLA_7SEG
    CALL    Enviar_Segmentos

    BANKSEL PORTA
    BSF     PORTA, 0            ; encender D0
    CALL    RETARDO_3MS
```

```
    ; 3. apagar displays
    BCF     PORTA, 0
    BCF     PORTA, 1

    ; 4. mostrar decenas
    BANKSEL decenas
    MOVLW   HIGH TABLA_7SEG
    MOVWF   PCLATH
    MOVF    decenas, W
    CALL    TABLA_7SEG
    CALL    Enviar_Segmentos

    BANKSEL PORTA
    BSF     PORTA, 1            ; encender D1
    CALL    RETARDO_3MS

; lectura del pulsador con antirrebote
Loop_Pulsador:
    BANKSEL PORTE
    BTFSC   PORTE, 2            ; si RE2 = 1, boton liberado
    GOTO    Boton_Liberado

    ; RE2 = 0: posible pulsacion
    BANKSEL ESTADO_BOTON
    BTFSC   ESTADO_BOTON, 0
    GOTO    Boton_Ya_Presionado

    ; confirmar cuatro muestras bajas consecutivas
    INCF    DEB_CONT, F
    MOVLW   d'4'
    SUBWF   DEB_CONT, W
    BTFSS   STATUS, Z
    GOTO    Seguir_Loop

    ; pulsacion confirmada: registrar una sola vez
    BSF     ESTADO_BOTON, 0
    CLRF    DEB_CONT

    ; incrementar contador binario (contador = contador + 1)
    BANKSEL contador
    INCF    contador, F
```

```
    ; verificar límite de 99 (volver a 00)
    MOVLW   d'100'
    SUBWF   contador, W
    BTFSC   STATUS, Z
    CLRF    contador

    ; mostrar valor binario en el banco de leds
    MOVF    contador, W
    MOVWF   PORTD

    ; realizar conversión de binario a BCD
    CALL    Convertir_Bin_Dec
    GOTO    Seguir_Loop

Boton_Ya_Presionado:
    ; no repetir el incremento mientras siga presionado
    ; reiniciar la cuenta de muestras de liberacion
    CLRF    DEB_CONT
    GOTO    Seguir_Loop

Boton_Liberado:
    BANKSEL ESTADO_BOTON
    BTFSS   ESTADO_BOTON, 0
    GOTO    Boton_Sin_Pulsar

    ; confirmar cuatro muestras altas consecutivas
    INCF    DEB_CONT, F
    MOVLW   d'4'
    SUBWF   DEB_CONT, W
    BTFSS   STATUS, Z
    GOTO    Seguir_Loop

    ; liberacion confirmada: habilitar otra pulsacion
    BCF     ESTADO_BOTON, 0
    CLRF    DEB_CONT
    GOTO    Seguir_Loop

Boton_Sin_Pulsar:
    ; sin pulsacion registrada: esperar al siguiente ciclo
    CLRF    DEB_CONT
```

```
Seguir_Loop:
    GOTO    Loop_Multiplexado  ; Retorna inmediatamente al refresco

; subrutina de conversion binario a decimal
Convertir_Bin_Dec:
    CLRF    decenas     ; inicia las decenas en 0
    MOVF    contador, W
    MOVWF   unidades

Resta_10:
    MOVLW   d'10'
    SUBWF   unidades, W         ; W = unidades - 10
    BTFSS   STATUS, C           ; verifica si el resultado es menor a 10
    GOTO    Fin_Conversion      ; si C = 0
    MOVWF   unidades            ; si C = 1, se guarda el resultado parcial
    INCF    decenas, F          ; incrementar el contador de decenas
    GOTO    Resta_10

Fin_Conversion:
    RETURN

; subrutina para la salida de los displays
Enviar_Segmentos:
    MOVWF   PATRON_TEMP
    BANKSEL PORTC
    MOVF    PORTC, W
    ANDLW   0xC0                ; conservar TX y RX
    MOVWF   PORTC_TEMP

    MOVF    PATRON_TEMP, W
    ANDLW   0x3F                ; conservar RC0 a RC5 (segmentos a-f)
    IORWF   PORTC_TEMP, W       ; combinar con RC0 a RC5 con TX y RX
    MOVWF   PORTC

    CLRF    PORTE_TEMP
    BTFSC   PATRON_TEMP, 6      ; verifica si RE0 (segmento g) debe encenderse
    BSF     PORTE_TEMP, 0       ; poner RE0 en alto

    BTFSC   PATRON_TEMP, 7      ;  verifica si RE1 (segmento dp) debe encenderse
    BSF     PORTE_TEMP, 1       ; poner RE1 en alto

    ; reserva el bit 2 del puerto E para no sobrescribir la lectura de RE2
```

```
    BANKSEL PORTE
    BTFSC   PORTE, 2
    BSF     PORTE_TEMP, 2

    MOVF    PORTE_TEMP, W
    MOVWF   PORTE
    RETURN

; subrutina de retardo de 3 ms
RETARDO_3MS:
    MOVLW   d'3'
    MOVWF   REG_MS
loop_ms:
    MOVLW   d'249'
    MOVWF   REG_US
loop_us:
    NOP
    DECFSZ  REG_US, F
    GOTO    loop_us
    DECFSZ  REG_MS, F
    GOTO    loop_ms
    RETURN

; tabla de decodificacion
    ORG     0x0300
TABLA_7SEG:
    ADDWF   PCL, F              ; suma el desplazamiento en W al PCL
    RETLW   0x3F                ; codigo para el 0 (abcdef)
    RETLW 0x06                  ; codigo para el 1 (bc)
    RETLW 0x5B                  ; codigo para el 2 (abdeg)
    RETLW 0x4F                  ; codigo para el 3 (abcdg)
    RETLW 0x66                  ; codigo para el 4 (bcfg)
    RETLW 0x6D                  ; codigo para el 5 (acdfg)
    RETLW 0x7D                  ; codigo para el 6 (acdefg)
    RETLW 0x07                  ; codigo para el 7 (abc)
    RETLW 0x7F                  ; codigo para el 8 (abcdefg)
    RETLW 0x6F                  ; codigo para el 9 (abcdfg)

    END
```

4. Pruebas y resultados

Con el fin de verificar el funcionamiento del circuito y del código, se probó el sistema de manera ordenada sobre la placa. El objetivo principal fue asegurar que respondiera correctamente según el tiempo que se mantuviera presionado el botón.

Dificultades: El principal problema en el funcionamiento fue, una mala conexion de los display, en el cual los digitos de las unidades, se veian en el lugar de las decenas y viceversa.

Prueba	Resultado esperado	Resultado observado
Accionamiento y antirrebote del pulsador	Cada pulsación válida debe producir un único incremento. La presión y la liberación deben confirmarse mediante cuatro muestras consecutivas del nivel lógico correspondiente. Mantener el botón presionado no debe generar incrementos adicionales.	Se verificó que cada pulsación válida del botón produce un único incremento del contador. Al mantener presionado el pulsador, el contador permanece en el mismo valor, evitando incrementos repetidos. Asimismo, la liberación del botón habilita una nueva pulsación después de confirmarse la estabilidad de la señal mediante cuatro muestras consecutivas. Durante estas comprobaciones, los displays continuaron actualizándose mediante el multiplexado, sin interrupciones perceptibles en la visualización.
Multiplexado y refresco de displays	Alternar la excitación de los dos dígitos con una frecuencia suficiente para evitar parpadeos visibles y apagar el dígito previo antes de cambiar los segmentos para impedir mezcla de cifras	Con un barrido adecuado, ambos displays presentaron un brillo homogéneo y continuo sin parpadeo perceptible. No se observó superposición de segmentos entre dígitos
Decodificación de símbolos (0 al 9)	La tabla de conversión en memoria de programa debe decodificar correctamente el valor binario de cada dígito en sus respectivos segmentos a-g.	Se comprobaron individualmente los diez símbolos (0 a 9) en ambos displays, visualizándose con total nitidez y correspondencia formal.
Transiciones críticas del contador	El contador debe realizar correctamente las transiciones decimales 09 $\rightarrow$ 10, 19 $\rightarrow$ 20 y el reinicio cíclico 99 $\rightarrow$ 00.	Se forzaron y observaron las transiciones 09 $\rightarrow$ 10, 19 $\rightarrow$ 20 y el retorno a 00 tras alcanzar 99, respondiendo de manera correcta y sin desfasajes en las cifras.
Correspondencia binaria en banco de LEDs	Los 8 LEDs conectados al puerto D deben reflejar simultáneamente el valor del contador en formato binario puro	Se constató la total sincronía entre el valor decimal exhibido en los displays y la representación binaria en los LEDs en todo el rango de conteo.

Tabla 3: *Resumen de las pruebas realizadas.*

5. Conclusiones

En este trabajo práctico se logró implementar con éxito el contador decimal de 00 a 99, cumpliendo con todos los objetivos y comprobaciones requeridas.

La técnica de multiplexado demostró ser muy eficiente, ya que permitió manejar dos displays utilizando un único bus de datos compartido para los segmentos, reduciendo la cantidad de pines necesarios del microcontrolador. Además, al ajustar el tiempo de encendido en 3 ms por display, se obtuvo una visualización continua, clara y sin parpadeos molestos.

Por último, se incorporó un mecanismo de antirrebote por software que diferencia la memoria del estado de la pulsación, implementada mediante `ESTADO_BOTON`, de la confirmación de estabilidad de la señal, realizada mediante `DEB_CONT`. El procedimiento requiere cuatro muestras consecutivas para confirmar tanto la presión como la liberación del pulsador, con el objetivo de evitar incrementos múltiples ocasionados por los rebotes mecánicos. Además, al integrar la comprobación del botón en el ciclo principal, se busca mantener el refresco de los displays sin recurrir a retardos bloqueantes prolongados.

Propuestas de Mejora y Optimización

Para futuras revisiones del diseño, se propone la siguiente mejora:

- **Optimización del Prototipo Físico (Hardware):** Dado que el circuito actual se montó de forma temporal sobre una protoboard, la principal mejora de hardware consiste en soldarlo en una placa multipropósito. Esto garantizaría la eliminación de fallos por falsos contactos mecánicos y reduciría las capacitancias parásitas.
- **Corrección del Antirrebote por Software:** Se modificó el código para mejorar la detección de las pulsaciones del botón. Se identificó que la variable `ESTADO_BOTON`, por sí sola, no garantiza la eliminación de los rebotes mecánicos, ya que una lectura momentánea de nivel alto puede habilitar nuevamente el conteo ante una posterior lectura de nivel bajo. Para solucionar esta limitación, se incorporó la variable `DEB_CONT`, que permite confirmar la estabilidad de la señal mediante cuatro muestras consecutivas, tanto al presionar como al liberar el pulsador. De esta manera, `ESTADO_BOTON` mantiene la función de memorizar si una pulsación ya fue registrada, mientras que `DEB_CONT` se encarga de verificar la estabilidad de la señal. Esta modificación busca evitar incrementos múltiples provocados por los rebotes mecánicos, manteniendo el refresco de los displays multiplexados sin introducir retardos bloqueantes prolongados.